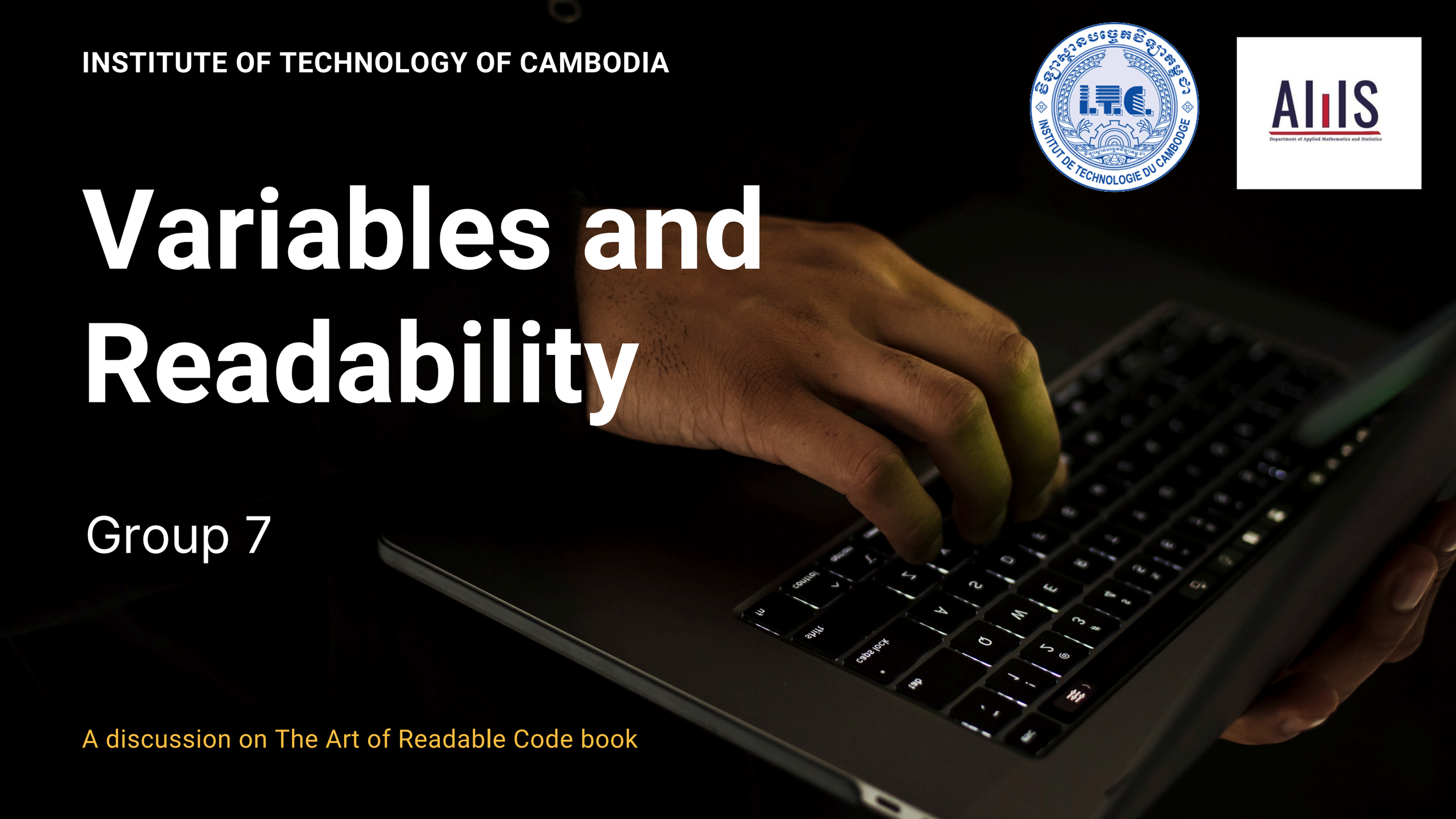
INSTITUTE OF TECHNOLOGY OF CAMBODIA

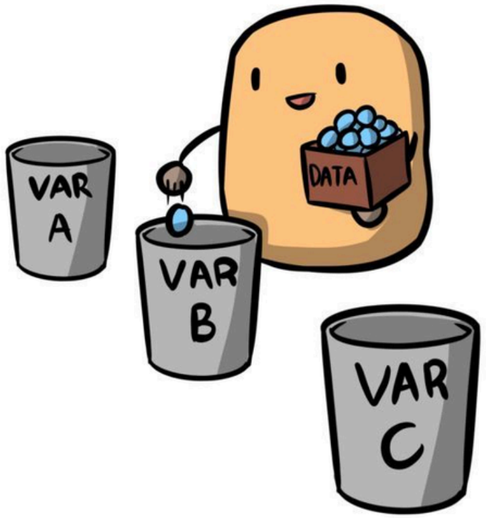


Variables and Readability

Group 7

A discussion on The Art of Readable Code book

Lesson 1: Eliminating Variables



Reducing mental clutter, shrinking scope, and improving program readability with Python examples.

Why Eliminate Variables?

Reduce Mental Overhead

Fewer variables mean less state to track simultaneously.

Shrink Variable Scope

Isolate variables so they do not linger across functions.

Execute & Exit Early

Process results instantly without storing intermediate data.

TECHNIQUE

Remove Useless Temporary Variables

Temporary variables often creep in during debugging or initial drafting. If a value is self-explanatory and used only once, assign or use it directly.

Bad • Extra Variable **# 'now' adds zero extra value now = datetime.datetime.now() root_message.last_view = now**

```
# 'now' adds zero extra value now =
datetime.datetime.now() root_message.last_view
= now
```

Good • Direct Call **# Self-describing & direct root_message.last_view = \ datetime.datetime.now()**

```
# Self-describing & direct
root_message.last_view = \
datetime.datetime.now()
```

Eliminate Intermediate Results

Instead of saving a calculation or lookup to a variable to use at the bottom of a function, process the result immediately and exit early using return or break.

BAD • STORING STATE

```
def remove_val(items, val):
    idx = None
    for i, item in enumerate(items):
        if item == val:
            idx = i
            break
    if idx is not None:
        items.pop(idx)
```

Good • Return Early

```
def remove_val(items, val):
    for i, item in enumerate(items):
        if item == val:
            items.pop(i)
            return # Complete & exi
```

Eliminate Control Flow Flags

Control flow variables are boolean flags (like done = False or found = True) used solely to direct loop execution. In Python, you can replace these with break, continue, or built-in loop mechanics (like for...else).

BAD • BOOLEAN FLAG

```
done = False
while condition and not done:
    # Process tasks...
    if should_stop:
        done = True
        continue
```

GOOD • STRUCTURED BREAK

```
while condition: # Process tasks... if
should_stop: break
```

Lesson 2: Reduce the Scope of Variables

Scope means the part of a program where a variable can be accessed.

The benefit of reduce the scope of Variables

- **Less to Remember**
- **Easier debugging**



The Comparison

```
discount = 20

def calculate():
    price = 100
    return price - discount
print(calculate())
```

✓ 0.0s

```
80
```

Before

Vs

```
def calculate():
    price = 100
    discount = 20
    return price - discount
print(calculate())
```

✓ 0.0s

```
80
```

After

Lesson 3: Prefer Write-Once Variables

What Are Write-Once Variables & Why Use Them?

- **Key Principle:** The more places a variable is manipulated, the harder it is to reason about its current value.
- **The Problem:** Variables that constantly change values force the reader to track their state line-by-line, causing cognitive overload ("mental baggage")
- **The Solution:**
  - Assign a variable its value once and keep it fixed.
  - Once defined, the reader doesn't need to re-check if the value was altered elsewhere



Code Implementation (Before vs. After)

Refactoring to Write-Once Variables

```
// ❌ BEFORE: Hard to read (Variable modified repeatedly)
var elem = getElement();
var found = false;

while (elem !== null) {
    if (elem.value === '') {
        found = true;       // State change #1
        break;
    }
    elem = getNextElement(); // State change #2
}
if (found) elem.value = "New Value";
```

```
// ✅ AFTER: Readable (Write-Once / Created fresh & assigned once)
for (var i = 1; true; i++) {
    const elem = getElement(i); // Assigned ONCE per iteration
    if (elem === null) return null;

    if (elem.value === '') {
        elem.value = "New Value";
        return elem;            // Handles result immediately!
    }
}
```

Takeaway: Fewer variable reassignments = fewer bugs and faster understanding

Lesson 4: A Final Example — Too Many Changing Variables

The Problem
Web page has serveral text input:

```
input1 → Dustin
input2 → Trevor
input3 → [ empty ]
input4 → Melissa
```

What is the problem
The code works, but we need to keep track of three variables for most of the function, and all of them are changed multiple times.

Goal:
Put a new value into the first empty input and return that input. If no empty input exists, return null.

Original Code

```
var found = false;
var i = 1;
var elem = document.getElementById('input' + i);

while (elem !== null) {
    if (elem.value === '') {
        found = true;
        break;
    }

    i++;
    elem = document.getElementById('input' + i);
}

if (found)
    elem.value = new_value;
```

```
3 variables to track

found → changes
i     → changes
elem  → changes
```

Refactoring the Variables

Improve code:

```
var setFirstEmptyInput = function (new_value) {

    for (var i = 1; true; i++) {

        var elem =
            document.getElementById('input' + i);

        if (elem === null)
            return null;

        if (elem.value === '') {
            elem.value = new_value;
            return elem;
        }
    }
};
```

What change?

1. **Remove found**
Handle the result immediately and return early.

2. **Use a for loop**
Keep i controlled in one place.

3. **Create elem inside the loop**
Smaller scope and effectively write-once during each iteration.

Before vs After

Then make a 2-column table.

Before	After
`found` exists and changes	removed
`i` defined separately and changes	controlled by `for` loop
`elem` reassigned many times	created inside loop


THANK
YOU